

# Introduction to Logical Arrays

## Part 1: What Are Logical Arrays?

Logical arrays are arrays of true/false values (1/0) used for indexing, filtering, and condition-based operations in MATLAB.

Create a logical array

```
logical_array = [true, false, true, false]
```

```
logical_array = 1x4 logical array  
    1     0     1     0
```

Logical arrays are often created using relational operations

```
A = [9, 2, 3, 4];  
B = [3, 2, 1, 4];  
L = A > B % Compare each element of A with B
```

```
L = 1x4 logical array  
    1     0     1     0
```

Logical arrays can be used to filter data

```
filtered_data = A(L) % Extract elements of A where L is true
```

```
filtered_data = 1x2  
    9     3
```

## Part 2: Logical Operations and Functions

Logical operators

```
L1 = [true, false, true];  
L2 = [false, true, true];
```

Logical AND (L1 & L2):

```
L1 & L2
```

```
ans = 1x3 logical array  
    0     0     1
```

Logical OR (L1 | L2):

```
L1 | L2
```

```
ans = 1x3 logical array  
    1     1     1
```

Logical NOT (~L1):

```
~L1
```

```
ans = 1x3 logical array
    0     1     0
```

Useful functions for logical arrays

**any:** Are any values true?

```
any(L1) % True if any element is true
```

```
ans = logical
      1
```

**all:** Are all values true?

```
all(L1) % True if all elements are true
```

```
ans = logical
      0
```

## Part 3: Logical Indexing

Create a numeric array

```
C = [10, 20, 30, 40, 50];
```

Create a logical array for indexing

```
L = C > 25
```

```
L = 1x5 logical array
    0     0     1     1     1
```

Use logical indexing to filter elements

```
filtered = C(L)
```

```
filtered = 1x3
    30     40     50
```

Modify elements using logical indexing

```
C(L) = C(L) * 2 % Double values greater than 25
```

```
C = 1x5
    10     20     60     80    100
```

## Part 4: Application: Getting Neighbors

Logical indexing can also be used to get the neighbors of a vertex from the adjacency matrix.

Adjacency Matrix

```
A = [0 1 0 0;
      1 0 1 1;
      0 1 0 1;
```

```
0 0 1 0];
```

Vertices

```
vertices = 1:4;
```

Pick a vertex

```
v = 2;
```

Get a logical array of every location in the v-th row of the adjacency matrix that is greater than 0.

```
is_neighbor_of_v = A(v,:) > 0
```

```
is_neighbor_of_v = 1x4 logical array  
1 0 1 1
```

```
neighbors_of_v = vertices(is_neighbor_of_v)
```

```
neighbors_of_v = 1x3  
1 3 4
```

## Part 5: Application: Cycle Detection

Recall: In the cycle detection algorithm, logical arrays are used to:

1. Identify nodes with certain properties (e.g., degree  $\leq 1$ ).
2. Track visited/permanent nodes.

Adjacency Matrix

```
A = [0 1 0 0;  
     1 0 1 0;  
     0 1 0 1;  
     0 0 1 0];
```

Step 1: Compute degrees of nodes

```
node_degrees = sum(A, 2) % Sum along rows
```

```
node_degrees = 4x1  
1  
2  
2  
1
```

Step 2: Identify nodes with degree  $\leq 1$  (logical array)

```
low_degree_nodes = node_degrees <= 1
```

```
low_degree_nodes = 4x1 logical array  
1  
0  
0  
1
```

Step 3: Remove low degree nodes from the adjacency matrix

Adjacency matrix before removal:

```
disp(A);
```

```
0    1    0    0
1    0    1    0
0    1    0    1
0    0    1    0
```

```
A(low_degree_nodes, :) = []; % Remove rows
A(:, low_degree_nodes) = []; % Remove columns
```

Adjacency matrix after removal:

```
disp(A);
```

```
0    1
1    0
```